

# **EXPERIMENT NO# 01**

## **OBJECTIVE:**

To understand the basic structure and functioning of a blockchain and implement a simple blockchain with multiple blocks using Python.

## **EQUIPMENT**

Computer with Python 3.x installed, Google Colab or Jupyter notebook for code execution and Internet connection to install required packages (if any).

## **THEORY**

Blockchain is a decentralized ledger which consists of blocks chained together via cryptographic hashes. Each block contains data, a timestamp, a nonce (for mining), and the hash of the previous block. Mining involves calculating a hash with a certain difficulty target (proof-of-work), ensuring tamper-resistance and security.

Mathematically, for block i:

$$\text{hash}_i = \text{SHA} - 256(\text{index} \parallel \text{timestamp} \parallel \text{data} \parallel \text{nonce} \parallel \text{previous\_hash})$$

where  $\parallel$  denotes concatenation.

## **EXPERIMENT AND PROGRAMMING**

### **OBSERVATION-01**

- Open the Google Colab
- Open the File
- Upload the Jupyter File from below link
- [https://github.com/niaz1971/Digital\\_Transformation\\_Industry\\_4\\_5/blob/main/PRACTICAL\\_WORK\\_BOOK/CREATE\\_BLOCK\\_CHAIN.ipynb](https://github.com/niaz1971/Digital_Transformation_Industry_4_5/blob/main/PRACTICAL_WORK_BOOK/CREATE_BLOCK_CHAIN.ipynb)

```
import hashlib
import datetime
import networkx as nx
import matplotlib.pyplot as plt

class Block:
    def __init__(self, index, timestamp, data, previous_hash):
        self.index = index
        self.timestamp = timestamp
        self.data = data
        self.previous_hash = previous_hash
        self.nonce = 0
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        content = str(self.index) + str(self.timestamp) +
str(self.data) + str(self.previous_hash) + str(self.nonce)
        return hashlib.sha256(content.encode()).hexdigest()

    def mine_block(self, difficulty):
        prefix = '0' * difficulty
        while self.hash[:difficulty] != prefix:
            self.nonce += 1
            self.hash = self.calculate_hash()

class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]
        self.difficulty = 4
```

```

def create_genesis_block(self):
    return Block(0, datetime.datetime.now(), "Genesis Block", "0")

def get_latest_block(self):
    return self.chain[-1]

def add_block(self, new_block):
    new_block.previous_hash = self.get_latest_block().hash
    new_block.mine_block(self.difficulty)
    self.chain.append(new_block)

def is_chain_valid(self):
    for i in range(1, len(self.chain)):
        curr = self.chain[i]
        prev = self.chain[i-1]

        if curr.hash != curr.calculate_hash():
            return False
        if curr.previous_hash != prev.hash:
            return False
    return True

def plot_blockchain(blockchain):
    G = nx.DiGraph()

    for block in blockchain.chain:
        label =
f"Index:{block.index}\nHash:{block.hash[:6]}...\nNonce:{block.nonce}"
        G.add_node(block.index, label=label)

    for i in range(1, len(blockchain.chain)):
        G.add_edge(blockchain.chain[i-1].index,
blockchain.chain[i].index)

    pos = nx.spring_layout(G, seed=42) # layout for neat
visualization
    labels = nx.get_node_attributes(G, 'label')
    plt.figure(figsize=(12,6))
    nx.draw(G, pos, with_labels=True, labels=labels, node_size=2500,
node_color='lightblue', font_size=8, font_weight='bold', arrows=True)
    plt.title("Blockchain Structure Visualization")
    plt.show()

# Main execution

my_blockchain = Blockchain()

for i in range(1, 11):
    print(f"Mining block {i}...")
    data = {"amount": i * 10}
    new_block = Block(i, datetime.datetime.now(), data, "")
    my_blockchain.add_block(new_block)

# Print blockchain details

```

```

for block in my_blockchain.chain:
    print(f"Index: {block.index}")
    print(f"Timestamp: {block.timestamp}")
    print(f>Data: {block.data}")
    print(f"Hash: {block.hash}")
    print(f"Previous Hash: {block.previous_hash}\n")

print("Is blockchain valid?", my_blockchain.is_chain_valid())

# Plot the blockchain structure
plot_blockchain(my_blockchain)

```

- Run the Code

## RESULT

Mining block 1...

Mining block 2...

Mining block 3...

Mining block 4...

Mining block 5...

Mining block 6...

Mining block 7...

Mining block 8...

Mining block 9...

Mining block 10...

Index: 0

Timestamp: 2025-10-30 17:59:46.296600

Data: Genesis Block

Hash: 18e46e25150c364f3f193ae04c0b9a35162d5521cd4cc4b71bd2055b08a91b3d

Previous Hash: 0

Index: 1

Timestamp: 2025-10-30 17:59:46.305104

Data: {'amount': 10}

Hash: 00007768f3adc0f93006a8d60e54badc5660ffdb82a53734a6e453b66ab0839b

Previous Hash: 18e46e25150c364f3f193ae04c0b9a35162d5521cd4cc4b71bd2055b08a91b3d

Index: 2

Timestamp: 2025-10-30 17:59:46.485511

Data: {'amount': 20}

Hash: 00003c19b5f2ec93479cf8709d6949cc56b1d3cb58d8b7590ed90ef0240b5c05

Previous Hash: 00007768f3adc0f93006a8d60e54badc5660ffdb82a53734a6e453b66ab0839b

Index: 3

Timestamp: 2025-10-30 17:59:46.595431

Data: {'amount': 30}

Hash: 0000d8379665798798fa28e252d7a88e55585f722e89c7de89dc6af1653dcee9

Previous Hash: 00003c19b5f2ec93479cf8709d6949cc56b1d3cb58d8b7590ed90ef0240b5c05

Index: 4

Timestamp: 2025-10-30 17:59:46.635753

Data: {'amount': 40}

Hash: 000021733073260c90dfb575c1ff04b57402b949c6f1ab67c65441ae1ac0c756

Previous Hash: 0000d8379665798798fa28e252d7a88e55585f722e89c7de89dc6af1653dcee9

Previous values are same

Index: 5  
Timestamp: 2025-10-30 17:59:47.316432  
Data: {'amount': 50}  
Hash: 0000a59c47d35db45e6182a36355ee28b95edbd46a4c6b9ee3e6d300257a6dbc  
Previous Hash: 000021733073260c90dfb575c1ff04b57402b949c6f1ab67c65441ae1ac0c756

Index: 6  
Timestamp: 2025-10-30 17:59:47.376522  
Data: {'amount': 60}  
Hash: 0000d3822060b73294b8acce620c6d334a96bd55c044dbe2fb2df43bba8b693d  
Previous Hash: 0000a59c47d35db45e6182a36355ee28b95edbd46a4c6b9ee3e6d300257a6dbc

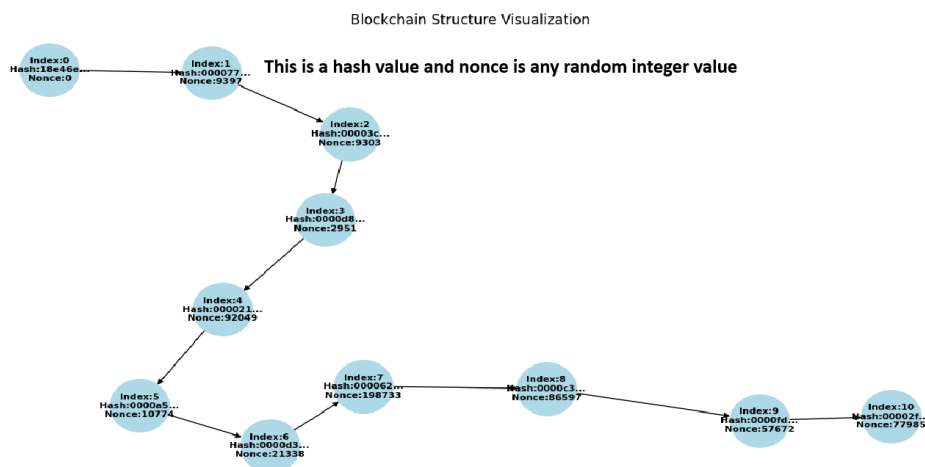
Index: 7  
Timestamp: 2025-10-30 17:59:47.479743  
Data: {'amount': 70}  
Hash: 0000622fa210051990be9b45061f3f2fd377b754685ee39a19174f9a2a79ca2c  
Previous Hash: 0000d3822060b73294b8acce620c6d334a96bd55c044dbe2fb2df43bba8b693d

Index: 8  
Timestamp: 2025-10-30 17:59:48.859041  
Data: {'amount': 80}  
Hash: 0000c37bde816055095fb41f169582241e1b1ad64f0264e23c47a4c08c62f6fe  
Previous Hash: 0000622fa210051990be9b45061f3f2fd377b754685ee39a19174f9a2a79ca2c

Index: 9  
Timestamp: 2025-10-30 17:59:49.067672  
Data: {'amount': 90}  
Hash: 0000fd2d1610c9d1f19b3bf0130aece7c15ad80cc3a566b8d99f5a3b2f54dacd  
Previous Hash: 0000c37bde816055095fb41f169582241e1b1ad64f0264e23c47a4c08c62f6fe

Index: 10  
Timestamp: 2025-10-30 17:59:49.208740  
Data: {'amount': 100}  
Hash: 00002f3a78c1749a559b32f683f44065c443fb0a43896b241ae02a8874c9db4d  
Previous Hash: 0000fd2d1610c9d1f19b3bf0130aece7c15ad80cc3a566b8d99f5a3b2f54dacd

Is blockchain valid? True



## OBSERVATION-02

- Open the Google Colab
- Open the File
- Upload the Jupyter File from below link
- [https://github.com/niaz1971/Digital\\_Transformation\\_Industry\\_4\\_5/blob/main/PRACTICAL\\_WORK\\_BOOK/CREATE\\_BLOCK\\_CHAIN\\_KHUZDAR\\_HOSPITAL.ipynb](https://github.com/niaz1971/Digital_Transformation_Industry_4_5/blob/main/PRACTICAL_WORK_BOOK/CREATE_BLOCK_CHAIN_KHUZDAR_HOSPITAL.ipynb)
- Run the Code

```

import hashlib
import datetime
import networkx as nx
import matplotlib.pyplot as plt

class Block:
    def __init__(self, index, timestamp, data, previous_hash):
        self.index = index
        self.timestamp = timestamp
        self.data = data
        self.previous_hash = previous_hash
        self.nonce = 0
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        content = str(self.index) + str(self.timestamp) + str(self.data) +
str(self.previous_hash) + str(self.nonce)
        return hashlib.sha256(content.encode()).hexdigest()

    def mine_block(self, difficulty):
        prefix = '0' * difficulty
        while self.hash[:difficulty] != prefix:
            self.nonce += 1
            self.hash = self.calculate_hash()

class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]
        self.difficulty = 4

    def create_genesis_block(self):
        return Block(0, datetime.datetime.now(), "Genesis Block", "0")

    def get_latest_block(self):
        return self.chain[-1]

    def add_block(self, new_block):
        new_block.previous_hash = self.get_latest_block().hash
        new_block.mine_block(self.difficulty)
        self.chain.append(new_block)

    def is_chain_valid(self):
        for i in range(1, len(self.chain)):
            curr = self.chain[i]
            prev = self.chain[i-1]
            if curr.hash != curr.calculate_hash() or curr.previous_hash !=
prev.hash:
                return False
        return True

    def plot_blockchain(blockchain):
        G = nx.DiGraph()
        for block in blockchain.chain:
            label = f"Index: {block.index}\nHash: {block.hash[:6]}...\nNonce:
{block.nonce}"

```

```

        G.add_node(block.index, label=label)
    for i in range(1, len(blockchain.chain)):
        G.add_edge(blockchain.chain[i-1].index, blockchain.chain[i].index)
    pos = nx.spring_layout(G, seed=42)
    labels = nx.get_node_attributes(G, 'label')
    plt.figure(figsize=(10,5))
    nx.draw(G, pos, with_labels=True, labels=labels, node_color='lightcyan',
node_size=2000, font_size=8, arrows=True)
    plt.title("Blockchain of Patient Records - Jhalawan Hospital Khuzdar")
    plt.show()

# Patient data for 3 blocks - Jhalawan Hospital Khuzdar
patient_data = [
    {"hospital": "Jhalawan Hospital Khuzdar", "patient_name": "Ali",
"diagnosis": "Flu"},
    {"hospital": "Jhalawan Hospital Khuzdar", "patient_name": "Muhammad",
"diagnosis": "Cold"},
    {"hospital": "Jhalawan Hospital Khuzdar", "patient_name": "Ahmed",
"diagnosis": "Diabetes"}
]

# Initialize blockchain and add blocks
blockchain = Blockchain()
for i, pdata in enumerate(patient_data, 1):
    print(f"Mining block {i} for patient {pdata['patient_name']}...")
    new_block = Block(i, datetime.datetime.now(), pdata, "")
    blockchain.add_block(new_block)

# Display blockchain details
for block in blockchain.chain:
    print(f"Index: {block.index} | Data: {block.data} | Hash: {block.hash}")

print("\nIs blockchain valid?", blockchain.is_chain_valid())

# Plot blockchain
plot_blockchain(blockchain)

```

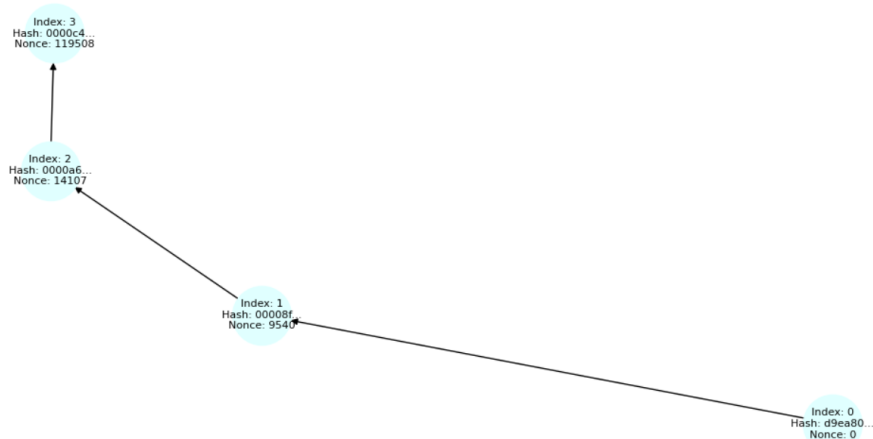
```

Mining block 1 for patient Ali...
Mining block 2 for patient Muhammad...
Mining block 3 for patient Ahmed...
Index: 0 | Data: Genesis Block | Hash: d9ea80c5075d46cb50261173595a51db40ad5e807f7f6468cff7f68746c02531
Index: 1 | Data: {'hospital': 'Jhalawan Hospital Khuzdar', 'patient_name': 'Ali', 'diagnosis': 'Flu'} | Hash: 00008fff48fdac1de7d984bc901c9c26f4bad7b96e62fcf735d24df721e209d3
Index: 2 | Data: {'hospital': 'Jhalawan Hospital Khuzdar', 'patient_name': 'Muhammad', 'diagnosis': 'Cold'} | Hash: 0000a666969aae4c753a4eb761fce9616e4ac3affe1f1c1fdd4f299822ddf893
Index: 3 | Data: {'hospital': 'Jhalawan Hospital Khuzdar', 'patient_name': 'Ahmed', 'diagnosis': 'Diabetes'} | Hash: 0000c48070ff922b19f40b186b816d9362a0f7e651f25ace0d2677a1a5a134f8

```

Is blockchain valid? True

Blockchain of Patient Records - Jhalawan Hospital Khuzdar



## **EXERCISE**

Write a Python and sets up a blockchain for five patients of any hospital near to your city, write five patients names, create the mining blocks, indexes, also verify is it blockchain valid? And finally draw blockchain structure.